

Adversarial Diffusion Distillation

Federico Raschiatore 2071132

October 1, 2024

1 Introduction

The Adversarial Diffusion Distillation (ADD) is a novel approach that aims to efficiently sample images in just a few steps. The idea is to combine the power of Adversarial networks and the distillation technique in order to synthesize high quality images in a small number of iterations.

2 Method description

The original model is composed by 3 parts:

- ADD-student[Sau23]: is a pretrained diffusion model that aims to synthesize the images in a few steps (1-4) starting from a noised version of the original image. So this model can use only from 1 to 4 iterations to denoise the input image. The parameters of this model are trainable.
- DM-teacher[Ho20]: is also a pretrained diffusion model but differently from the student, it can use a larger number of iterations to denoise the input. The parameters in this case are fixed.
- ViT Discriminator[Dos21]: is a pretrained Vision Transformer that is adapted to use as a discriminator with trainable parameters.

The training procedure is the following: starting from a real image, using a forward process, it introduces noise in the image obtaining a noised version x_s of the original image. The noised version is used as a starting point for the ADD-student model and using a denoising process it yields to a new image x_θ . This image is used both as an input to the discriminator model and as input to the teacher model. The discriminator model is used to discriminate between real images and generated images and is used to compute the adversarial loss as follows:

$$L_{adv}^D(\hat{x}_\theta(x_s, s), \phi) = E_{x_0} \left[\sum_k \max(0, 1 - D_{\phi, k}(F_k(x_0))) + \gamma R_1(\phi) \right] + E_{\hat{x}_\theta} \left[\sum_k \max(0, 1 + D_{\phi, k}(F_k(\hat{x}_\theta))) \right] \quad (1)$$

The teacher model instead repeats the noising and denoising process as we have seen before for the student model and we compute the distillation loss as follows:

$$L_{\text{distill}}(\hat{x}_\theta(x_s, s), \psi) = E_{t, \epsilon'} [c(t) \cdot d(\hat{x}_\theta, \hat{x}_\psi(s, g(\hat{x}_\theta, t); t))] \quad (2)$$

The adversarial loss and the distillation loss are used to compute the total loss using a multiplication factor λ to balance the two losses:

$$L_{\text{total}} = L_{adv}^D(\hat{x}_\theta(x_s, s), \phi) + \lambda L_{\text{distill}}(\hat{x}_\theta(x_s, s), \psi) \quad (3)$$

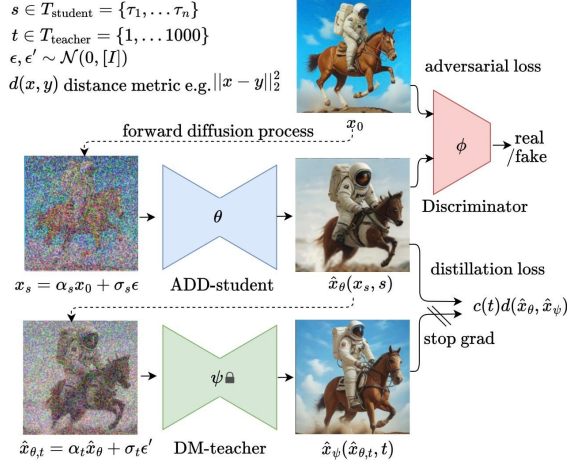


Figure 1: Adversarial diffusion distillation procedure

3 Datasets

For this project I used 2 different dataset

- **MNIST** is a dataset containing images of hand written numbers. I tested the model using a fixed image size of 32x32.

The represented images are relatively simple so i choosed this dataset in order to compare the performances when the model deal with simple images.

- **FER2013** is a dataset containing human faces that express different emotions. I have tried to work in using 3 channels but the model became too heavy and the training too slow so I decided to use only 1 channel in order to obtain good results in a relatively small time.

I choose this dataset in order to observe how the model performs when it have to deal with images full of details (a human face).

In both cases i decided to operate on a mini - batch because the datasets are very huge and the training time became very computationally expensive.

4 Training

The training for the diffusion model is done using 500 timesteps, 50 training epochs, an Adam optimizer with learning rate of 1e-3 and a scheduler with a patience of 10 epochs and a factor of 0.1

The training of the ViT Discriminator model is done using 1 epochs for the training and this is due to the fact that it learn very quickly for this reason i preferred to fine tune the model directly during the ADD training

The ADD training procedure is splitted in 2 different parts:

- **Pretraining** This phase is crucial for providing the student model with an initial fine-tuning to accelerate the ADD-training procedure and prevent mode collapse and non-convergence. Based on extensive testing, I have observed that the ADD-training is highly sensitive and prone to falling into local minima. Therefore, I decided to introduce this pretraining phase.

The pretraining consists of a purely distillation-based training in which both the student and teacher models start from the same noisy image and attempt to denoise it. The loss is computed between the denoised images produced by the teacher and the student models.

This approach enables the student model to learn the process of denoising an image. While this is beneficial for understanding the overall structure and shape of the image, it is not as effective for capturing fine details.

The number of epochs for this pretraining procedure is kept deliberately low. Through experimentation, I found that after 10 epochs, the improvements in the quality of the produced images were negligible.

Hyperparameters:

- Learning rate: 1e-3
- Patience: 10
- Epochs : 20

- **ADD - training** This phase is the training of the original paper where starting from the pretrained models, the adversarial diffusion distillation is done, as specified above. In this phase both the student generator and the discriminator are trained. To balance the training of the two models I decided to update the parameters of the discriminator model only after 10 steps.

I also decided to add a regularization term in order to avoid mode collapse. The regularization strategy that i decided to use is the following:

$$\mathcal{L}_{\text{diversity}} = -\frac{1}{N} \sum_{i=1}^N |x_i - \mu| \quad (4)$$

Hyperparameters:

- Learning rate: 1e-3
- Patience: 15
- Epochs : 50

5 Results

In this section, you will find a qualitative comparison of the results along with the FID values obtained from my models.

The results are significantly influenced by hardware limitations, particularly the constraints of GPU size and the available GPU time provided by Google Colab.

To address these limitations, both models required multiple training cycles. During each cycle, I saved the model weights and reloaded them to continue training.

For the MNIST Model the results are similar to the original distribution data, indicating successful training. This is probably due the simplicity of the represented images and the lack of details.

For the FER Model the results are much worse compared to the original dataset, especially regarding details. In fact as you can see below the images appears like shadows and the model has not been able to represents details or facial expressions. This is mainly due to the low computational power available.

For both models, I experimented with various batch sizes and hidden dimensions. The results presented here are based on the best hyperparameters selected through these experiments.

5.1 MNIST

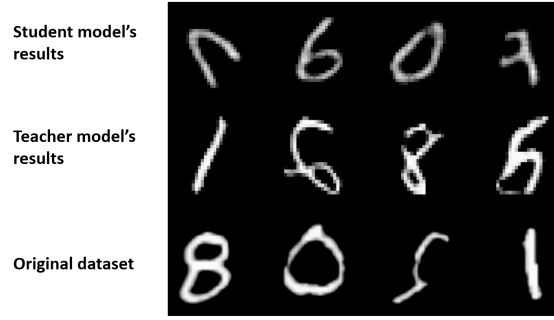


Figure 2: Qualitative comparison

Model	FID
Student model	102
Teacher mdoel	58

Table 1: FID values referring to the original dataset, the teacher model and the student model

5.2 FER

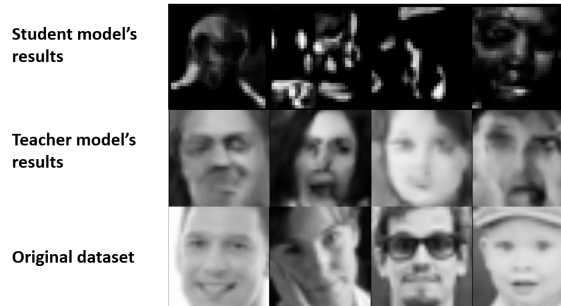


Figure 3: Qualitative comparison

Model	FID
Student model	206
Teacher mdoel	94

Table 2: FID values referring to the original dataset, the teacher model and the student model

6 Limitations

The limitations of my project were primarily due to the limited computational power available. I trained the model using Google Colab's GPU, which required a significant amount of time. As a result, I often had to save the weights and resume training the next day or switch to a different account.

I also experimented with pretrained diffusion models but found that student training was very slow due to the large number of parameters involved. Consequently, I decided to build and train my own

models to keep the number of weights as small as possible, though this made the training process quite slow.

Another challenge was balancing the hyperparameters, such as the number of teacher timesteps or the amount of noise introduced into the original image and the student denoised image. Balancing the learning rate was particularly difficult, so I implemented a scheduler to progressively reduce it over time.

7 Code Instructions

To run the code, follow these steps:

- Open the notebook file .ipynb
- Specify the Dataset: Choose the dataset you want to test the model on. You can select either MNIST or FER.
- Download Pretrained Model: If you want to use the pretrained models, download them from the project's GitHub repository. These weights can be used to test the models without retraining.
- Retrain the Model: If you prefer to train the models from scratch, do not import the pretrained models.

To test the models using the pretrained weights, simply run the test section in the notebook and skip the training sections.

In this case, you need to execute all the training sections in the notebook to train the models anew.

By following these instructions, you can either test the models with pretrained weights or retrain them from scratch as per your requirements.

8 Conclusion

In conclusion, the ADD framework shows great potential for efficient image synthesis, but the training process is very difficult and very sensible. It also needs a big computational power in order to ensure good results in particular for images with a lot of details and features to learn. I think is a great idea in order to produce images in a reasonable time but the training from scratch is very difficult and the convergence is not easy to reach.

References

- [Dos21] Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. 2021.
- [Ho20] Jonathan Ho. Denoising diffusion probabilistic models. 2020.
- [Sau23] Axel Sauer. Adversarial diffusion distillation. 2023.